

Lecture 5: cancellation and interruption

concurrent state machine, forced cancellation, cooperative cancellation, cancellation token, interruption, timeouts

Alexander Filatov
filatovaur@gmail.com

<https://github.com/Svazars/parallel-programming/blob/main/slides/pdf/l5.pdf>

In previous episodes

Basic tools for thread coordination

- Mutual exclusion
- Signalling: one-to-one and broadcast
- Group-level concurrency
- Pooling of computation agents and separating task from particular executor

Concurrent data structures:

- Lock, Condition, Monitor, CountdownLatch, Semaphore, ReadWriteLock

Thread-specific data could be used for caching:

- replication, snapshotting

Be aware of

- Race conditions, data races, visibility, concurrent consistency
- Progress guarantees: deadlock, priority inversion, fairness, resource-constrained deadlock
- Logical errors related to signalling: lost signal, spurious wakeup, predicate invalidation
- Performance: thundering herd, oversubscription

Lecture plan

- 1 Thread-safe counters
- 2 Cancellation
 - Cancelling Future
 - Forced cancellation
 - Cooperative cancellation: cancellation token
- 3 Interruption
 - Interruption points
 - Design decisions
 - Timeouts
- 4 Summary

Lecture plan

1 Thread-safe counters

2 Cancellation

- Cancelling Future
- Forced cancellation
- Cooperative cancellation: cancellation token

3 Interruption

- Interruption points
- Design decisions
- Timeouts

4 Summary

Synchronized counter

```
final class SynchronizedCounter {  
    private long data;  
    public SynchronizedCounter(long initial) {  
        data = initial;  
    }  
    public synchronized long get() {  
        return data;  
    }  
    public synchronized void add(long delta) {  
        data += delta;  
    }  
}
```

Read-mostly counter

```
final class ReadMostlyCounter {  
    private long data;  
    private ReadWriteLock rw = new ReentrantReadWriteLock();  
    public ReadMostlyCounter(long initial) { data = initial; }  
    public long get() {  
        rw.readLock().lock();  
        try { return data; }  
        finally { rw.readLock().unlock(); }  
    }  
    public void add(long delta) {  
        rw.writeLock().lock();  
        try { data += delta; }  
        finally { rw.writeLock().unlock(); }  
    }  
}
```

Write-mostly counter: lock splitting

```
final class SynchronizedCounter {  
    private long data;  
    public SynchronizedCounter(long initial) {  
        data = initial;  
    }  
    public synchronized long get() {  
        return data;  
    }  
    public synchronized void add(long delta) { // <----- lock splitting  
        data += delta;  
    }  
}
```

Reminder: Homework Task 2.4 (Medium)

Write-mostly counter: distributed counter

Distributed counter:

- N independent long slots – 1 slot per thread
- Every user of distributed counter should register itself
- Thread adds to its own slot (no contention)
- Thread gets from all slots (could take time)

Question time

Question: "1 slot per thread". Which Java class should I use?



Question time

Question: "user of distributed counter should register itself". Should it be reenterable operation?



Question time

Question:

- Thread adds to its own slot (no contention)
- Thread gets from all slots (could take time)

Do you know data structure that divides threads into two groups: "many simultaneous" and "only one"?



Distributed counter: thread-private slots

- N independent long slots – 1 slot per thread

```
class DistributedCounter {  
    class Slot { long data; }  
    static final ThreadLocal<Slot> mySlot = new ThreadLocal<>() {  
        protected Slot initialValue() { return null; }  
    };  
    boolean thisThreadIsRegistered() {  
        return mySlot.get() != null;  
    }  
    void ensureRegistered() {  
        if (!thisThreadIsRegistered()) { mySlot.set(new Slot()); }  
    }  
}
```

Distributed counter: thread-private slots

- N independent long slots – 1 slot per thread

```
class DistributedCounter {  
    class Slot { long data; }  
    static final ThreadLocal<Slot> mySlot = new ThreadLocal<>() {  
        protected Slot initialValue() { return null; }  
    };  
    boolean thisThreadIsRegistered() {  
        return mySlot.get() != null;  
    }  
    void ensureRegistered() {  
        if (!thisThreadIsRegistered()) { mySlot.set(new Slot()); }  
    }  
}
```

- Thread-safe?

Distributed counter: thread-specific lazy initialization

- Every user of distributed counter should register itself

```
class DistributedCounter {  
    ...  
    void add(long delta) {  
        ensureRegistered();  
        ???  
        ???  
        ???  
    }  
}
```

Distributed counter: scalable add

- Thread adds to its own slot (no contention)

```
class DistributedCounter {  
    final ReadWriteLock rw = new ReentrantReadWriteLock();  
    void add(long delta) {  
        ensureRegistered();  
        ???  
        ???  
        ???  
    }  
}
```

Distributed counter: scalable add

- Thread adds to its own slot (no contention)

```
class DistributedCounter {  
    final ReadWriteLock rw = new ReentrantReadWriteLock();  
    void add(long delta) {  
        ensureRegistered();  
        rw.readLock().lock();  
        try { mySlot.get().data++;}  
        finally { rw.readLock().unlock(); }  
    }  
}
```


Distributed counter: consistent get

- Thread gets from all slots (could take time)

```
class DistributedCounter {  
    final ReadWriteLock rw = new ReentrantReadWriteLock();  
    long get() {  
        ???  
        ???  
        ???  
    }  
}
```

Distributed counter: consistent get

- Thread gets from all slots (could take time)

```
class DistributedCounter {  
    final ReadWriteLock rw = new ReentrantReadWriteLock();  
    long get() { rw.writeLock().lock();  
                try { ??? }  
                finally { rw.writeLock().unlock(); }  
            }  
}
```

Distributed counter: consistent get

- Thread gets from all slots (could take time)

```
class DistributedCounter {
    final ReadWriteLock rw = new ReentrantReadWriteLock();
    long get() { rw.writeLock().lock();
                try { ??? }
                finally { rw.writeLock().unlock(); }
            }
    final List<Slot> lst = new ArrayList<>();
    void ensureRegistered() {
        if (thisThreadIsRegistered()) { return; }
        mySlot.set(new Slot());
        rw.writeLock().lock();
        try { lst.add(mySlot.get()); } finally { rw.writeLock().unlock(); }
    }
}
```

Distributed counter: consistent get

- Thread gets from all slots (could take time)

```
class DistributedCounter {
    final ReadWriteLock rw = new ReentrantReadWriteLock();
    long get() { rw.writeLock().lock(); long result = 0;
        try { for (var slot: lst) { result += slot.data; } }
        finally { rw.writeLock().unlock(); }
        return result; }
    final List<Slot> lst = new ArrayList<>();
    void ensureRegistered() {
        if (thisThreadIsRegistered()) { return; }
        mySlot.set(new Slot());
        rw.writeLock().lock();
        try { lst.add(mySlot.get()); } finally { rw.writeLock().unlock(); }
    }
}
```

Distributed counter: takeaways

- Read-mostly and write-mostly designs could be quite different
- `ThreadLocal` helps to organize uncontended execution
- `ReadWriteLock` splits threads into two groups
 - Not necessarily readers and writers!

Distributed counter: takeaways

- Read-mostly and write-mostly designs could be quite different
- `ThreadLocal` helps to organize uncontended execution
- `ReadWriteLock` splits threads into two groups
 - Not necessarily readers and writers!

Teaser

Distributed counter: takeaways

- Read-mostly and write-mostly designs could be quite different
- `ThreadLocal` helps to organize uncontended execution
- `ReadWriteLock` splits threads into two groups
 - Not necessarily readers and writers!

Teaser

- Replication + Snapshotting + Distributed systems = Headache

Distributed counter: takeaways

- Read-mostly and write-mostly designs could be quite different
- `ThreadLocal` helps to organize uncontended execution
- `ReadWriteLock` splits threads into two groups
 - Not necessarily readers and writers!

Teaser

- Replication + Snapshotting + Distributed systems = Headache
 - Wait-free snapshots (Lecture 8)
 - ABA problem (Lecture 9)
 - Replication on CPU level (Lecture 10)

Question time

Question: We know so much ways to coordinate multiple threads:

- Start threads with identical environment, eagerly or on-demand
 - How to achieve this?



Question time

Question: We know so much ways to coordinate multiple threads:

- Start threads, eagerly or on-demand (ThreadFactory, ExecutorService)



Question time

Question: We know so much ways to coordinate multiple threads:

- Start threads, eagerly or on-demand (`ThreadFactory`, `ExecutorService`)
- Handle contention with mutexes
 - What are the main design decisions that are associated with mutexes?



Question time

Question: We know so much ways to coordinate multiple threads:

- Start threads, eagerly or on-demand (`ThreadFactory`, `ExecutorService`)
- Handle contention with mutexes (reentrant, fair, read-write)



Question time

Question: We know so much ways to coordinate multiple threads:

- Start threads, eagerly or on-demand (ThreadFactory, ExecutorService)
- Handle contention with mutexes (reentrant, fair, read-write)
- Coordinate groups of threads



Question time

Question: We know so much ways to coordinate multiple threads:

- Start threads, eagerly or on-demand (`ThreadFactory`, `ExecutorService`)
- Handle contention with mutexes (reentrant, fair, read-write)
- Coordinate groups of threads (`CountDownLatch`, `Semaphore`)



Question time

Question: We know so much ways to coordinate multiple threads:

- Start threads, eagerly or on-demand (ThreadFactory, ExecutorService)
- Handle contention with mutexes (reentrant, fair, read-write)
- Coordinate groups of threads (CountDownLatch, Semaphore)
- Abstract asynchronous result from computation



Question time

Question: We know so much ways to coordinate multiple threads:

- Start threads, eagerly or on-demand (ThreadFactory, ExecutorService)
- Handle contention with mutexes (reentrant, fair, read-write)
- Coordinate groups of threads (CountDownLatch, Semaphore)
- Abstract asynchronous result from computation (Future)



Question time

Question: We know so much ways to coordinate multiple threads:

- Start threads, eagerly or on-demand (`ThreadFactory`, `ExecutorService`)
- Handle contention with mutexes (reentrant, fair, read-write)
- Coordinate groups of threads (`CountDownLatch`, `Semaphore`)
- Abstract asynchronous result from computation (`Future`)
 - ... or stream of results



Question time

Question: We know so much ways to coordinate multiple threads:

- Start threads, eagerly or on-demand (`ThreadFactory`, `ExecutorService`)
- Handle contention with mutexes (reentrant, fair, read-write)
- Coordinate groups of threads (`CountDownLatch`, `Semaphore`)
- Abstract asynchronous result from computation (`Future`)
 - ... or stream of results (`CompletionService`)



Question time

Question: We know so much ways to coordinate multiple threads:

- Start threads, eagerly or on-demand (`ThreadFactory`, `ExecutorService`)
- Handle contention with mutexes (reentrant, fair, read-write)
- Coordinate groups of threads (`CountDownLatch`, `Semaphore`)
- Abstract asynchronous result(s) from computation (`Future`, `CompletionService`)



Question time

Question: We know so much ways to coordinate multiple threads:

- Start threads, eagerly or on-demand (`ThreadFactory`, `ExecutorService`)
- Handle contention with mutexes (reentrant, fair, read-write)
- Coordinate groups of threads (`CountDownLatch`, `Semaphore`)
- Abstract asynchronous result(s) from computation (`Future`, `CompletionService`)
- Monitor/debug multithreaded software



Question time

Question: We know so much ways to coordinate multiple threads:

- Start threads, eagerly or on-demand (`ThreadFactory`, `ExecutorService`)
- Handle contention with mutexes (reentrant, fair, read-write)
- Coordinate groups of threads (`CountDownLatch`, `Semaphore`)
- Abstract asynchronous result(s) from computation (`Future`, `CompletionService`)
- Monitor/debug multithreaded software (`jstack`)



Question time

Question: We know so much ways to coordinate multiple threads:

- Start threads, eagerly or on-demand (ThreadFactory, ExecutorService)
- Handle contention with mutexes (reentrant, fair, read-write)
- Coordinate groups of threads (CountDownLatch, Semaphore)
- Abstract asynchronous result(s) from computation (Future, CompletionService)
- Monitor/debug multithreaded software (jstack)

We can **start** and **control** concurrent computation till the **termination**.

What else do we need?



Lecture plan

1 Thread-safe counters

2 Cancellation

- Cancelling Future
- Forced cancellation
- Cooperative cancellation: cancellation token

3 Interruption

- Interruption points
- Design decisions
- Timeouts

4 Summary

Cancellation

Program initiated some task and then

- user is no more interested in the result
- expected result is known to be out-of-date
- a lot of tasks with higher priority appeared
- program intentionally started several identical tasks¹

so we need to

¹<https://cacm.acm.org/research/the-tail-at-scale/>

Cancellation

Program initiated some task and then

- user is no more interested in the result
- expected result is known to be out-of-date
- a lot of tasks with higher priority appeared
- program intentionally started several identical tasks¹

so we need to

- **cancel** not-yet-started task; or

¹<https://cacm.acm.org/research/the-tail-at-scale/>

Cancellation

Program initiated some task and then

- user is no more interested in the result
- expected result is known to be out-of-date
- a lot of tasks with higher priority appeared
- program intentionally started several identical tasks¹

so we need to

- **cancel** not-yet-started task; or
- **interrupt** already running task

to save CPU/RAM/network bandwidth/...

¹<https://cacm.acm.org/research/the-tail-at-scale/>

Question time

Question: Is it always a good idea to **cancel** not-yet-started task when the result is not needed?



Future states

```
enum FutureState { Created, Submitted, Running, Canceled, Terminated }
// +-----+      +-----+      +-----+      +-----+
// | Created | -> | Submitted | -> | Running | -> | Terminated |
// +-----+      +-----+      +-----+      +-----+
//                |
//                V
//                +-----+
//                | Canceled |
//                +-----+
```

Future states

<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/CancellationException.html>

```
enum FutureState { Created, Submitted, Running, Canceled, Terminated }
class MyFuture<V> { FutureState state; MyFuture() { this.state = Created; }
    synchronized boolean tryCancel() {
        assert state != Created;
        if (state == Running || state == Terminated) { return false; }
        if (state == Canceled) { return true; }
        assert state == Submitted;
        state = Canceled; return true;
    }
    V get() throws ExecutionException {
        synchronized(this) {
            if (state == Canceled) { throw new CancellationException(); }
            ...
        }
    }
}
```

Future states

```
enum FutureState { Created, Submitted, Running, Canceled, Terminated }  
class MyExecutorService {  
    void submit(Callable<V> c) {  
        var f = new MyFuture<>(c); f.state = Submitted; blockingQueue.add(f);  
    }  
    void worker(MyFuture f) {  
        synchronized(f) {  
            if (f.state == Canceled) { handleCancel(f); return; }  
            assert f.state == Submitted;  
            f.state = Running;  
        }  
        try { doComputation(f); }  
        finally { synchronized(f) { f.state = Terminated; } }  
    }  
}
```

Pattern: concurrent state machine

- At any moment, state of concurrent object is explicitly defined
- All transitions happen atomically (e.g. under Lock)
- Every state defines **allowed** and **forbidden** operations

Pattern: concurrent state machine

- At any moment, state of concurrent object is explicitly defined
- All transitions happen atomically (e.g. under Lock)
- Every state defines **allowed** and **forbidden** operations

Advantages:

- Easy to use
- Simple to implement
- Trivial to debug using logging

Pattern: concurrent state machine

- At any moment, state of concurrent object is explicitly defined
- All transitions happen atomically (e.g. under Lock)
- Every state defines **allowed** and **forbidden** operations

Advantages:

- Easy to use
- Simple to implement
- Trivial to debug using logging

Disadvantages:

- *Could* limit parallelism
- Some concurrent objects have infinite amount of states

Pattern: concurrent state machine

- At any moment, state of concurrent object is explicitly defined
- All transitions happen atomically (e.g. under Lock)
- Every state defines **allowed** and **forbidden** operations

Advantages:

- Easy to use
- Simple to implement
- Trivial to debug using logging

Disadvantages:

- *Could* limit parallelism
- Some concurrent objects have infinite amount of states

Tricks:

- use equivalence classes as "states" (concurrent queue is EMPTY, NON_EMPTY, FULL)
- create a separate "control structure" for accessing your inner data (Executor.submit vs. Executor.taskQueue.add)

Pattern: cancellation of not-yet-processed tasks

- Many threads create workload (stream of tasks)
- Many threads process data (execute tasks)

How to find "submitted but not yet started" tasks?

- Inspect thread-safe "submission queue"

Pattern: cancellation of not-yet-processed tasks

- Many threads create workload (stream of tasks)
- Many threads process data (execute tasks)

How to find "submitted but not yet started" tasks?

- Inspect thread-safe "submission queue"
- What if there are many queues?

Do not try to control submission protocol, operate on task level

Disadvantages:

- more synchronized operations on fast-path
- quite complicated if you have different Future objects that represent the same task
 - remember CompletionService and AlreadyCompletedFuture?

Concurrent state machine training

<https://github.com/Svazars/parallel-programming/tree/main/hw/block1/5.1>

Homework

Task 5.1

- *Implement `ExecutorService` with `Ready`, `ShutDown`, `Terminated` life cycle*

Question time

Question: What should I do to stop already running task?



Lecture plan

1 Thread-safe counters

2 Cancellation

- Cancelling Future
- **Forced cancellation**
- Cooperative cancellation: cancellation token

3 Interruption

- Interruption points
- Design decisions
- Timeouts

4 Summary

Forced cancellation

Idea

Operation systems allow killing processes and threads:

- `kill -9 <PID>`²
- `int pthread_kill(pthread_t thread, int sig)`³

²<https://man7.org/linux/man-pages/man1/kill.1.html>

³https://man7.org/linux/man-pages/man3/pthread_kill.3.html

Forced cancellation

Idea

Operation systems allow killing processes and threads:

- `kill -9 <PID>`²
- `int pthread_kill(pthread_t thread, int sig)`³

Not exactly. They provide mechanism to *send a signal* to running thread which *asynchronously* "transfer" thread execution flow into signal handler.

² <https://man7.org/linux/man-pages/man1/kill.1.html>

³ https://man7.org/linux/man-pages/man3/pthread_kill.3.html

Forced cancellation

Idea

Operation systems allow killing processes and threads:

- `kill -9 <PID>`²
- `int pthread_kill(pthread_t thread, int sig)`³

Not exactly. They provide mechanism to *send a signal* to running thread which *asynchronously* "transfer" thread execution flow into signal handler.

By default, unhandled "important" signal (e.g. SIGKILL) will stop thread execution.

²<https://man7.org/linux/man-pages/man1/kill.1.html>

³https://man7.org/linux/man-pages/man3/pthread_kill.3.html

Forced cancellation

Idea

Operation systems allow killing processes and threads:

- `kill -9 <PID>`²
- `int pthread_kill(pthread_t thread, int sig)`³

Not exactly. They provide mechanism to *send a signal* to running thread which *asynchronously* "transfer" thread execution flow into signal handler.

By default, unhandled "important" signal (e.g. SIGKILL) will stop thread execution.

Some concurrent systems also provide facilities for forced asynchronous thread cancellation⁴.

² <https://man7.org/linux/man-pages/man1/kill.1.html>

³ https://man7.org/linux/man-pages/man3/pthread_kill.3.html

⁴ [https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#stop\(\)](https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#stop())

Question time

Question: Terminate running thread at arbitrary execution point: could you predict any problems?



Forced cancellation

Problems

Program could be stopped at arbitrary point:

- In the middle of file writing (corrupted data)
- In the middle of DB connect (lost data)
- In the middle of mutex critical section (lock leakage, deadlock)

Forced cancellation

Problems

Program could be stopped at arbitrary point:

- In the middle of file writing (corrupted data)
- In the middle of DB connect (lost data)
- In the middle of mutex critical section (lock leakage, deadlock)

Common problems:

- inconsistent global state
- non-recoverable data corruption

Forced cancellation

Problems

Program could be stopped at arbitrary point:

- In the middle of file writing (corrupted data)
- In the middle of DB connect (lost data)
- In the middle of mutex critical section (lock leakage, deadlock)

Common problems:

- inconsistent global state
- non-recoverable data corruption

Additional level of complexity: cancellation **inside** cancellation handler.

Forced cancellation

Problems

Program could be stopped at arbitrary point:

- In the middle of file writing (corrupted data)
- In the middle of DB connect (lost data)
- In the middle of mutex critical section (lock leakage, deadlock)

Common problems:

- inconsistent global state
- non-recoverable data corruption

Additional level of complexity: cancellation **inside** cancellation handler.

Conclusion: forced cancellation in concurrent systems is **bad design**⁵

⁵ <https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/doc-files/threadPrimitiveDeprecation.html>

Lecture plan

1 Thread-safe counters

2 Cancellation

- Cancelling Future
- Forced cancellation
- Cooperative cancellation: cancellation token

3 Interruption

- Interruption points
- Design decisions
- Timeouts

4 Summary

Cooperative cancellation

Idea

- Thread A initiates cancellation
- Thread B performs cancellation handler
- Thread A finishes cancellation protocol

Cooperative cancellation

Idea

- Thread A initiates cancellation
- Thread B performs cancellation handler
- Thread A finishes cancellation protocol

Design challenges:

- How to pass information from one thread to another
 - reasonable response time vs. non-prohibitive throughput overheads
- When to handle cancellation request
 - immediately, even inside of "critical code path"
 - defer processing
 - ordering among received requests
 - deduplication/queueing of requests
- What to do after cancellation handling
 - Send confirmation response
 - Stop thread/throw exception/continue execution

Cancellation token

```
final class CancellationToken {  
    private boolean canceled = false;  
    public synchronized boolean isCanceled() { return this.canceled; }  
    public synchronized void cancel() { this.canceled = true; }  
}
```

Cancellation token

```
final class CancellationToken {  
    private boolean canceled = false;  
    public synchronized boolean isCanceled() { return this.canceled; }  
    public synchronized void cancel() { this.canceled = true; }  
}
```

- How to pass information from one thread/task to another: lock + state variable
- When to handle cancellation request: when thread/task is ready to cooperate
- What to do after cancellation handling: up to receiver

Cancellation token

```
final class CancellationToken {  
    private boolean canceled = false;  
    public synchronized boolean isCanceled() { return this.canceled; }  
    public synchronized void cancel() { this.canceled = true; }  
}
```

- How to pass information from one thread/task to another: lock + state variable
- When to handle cancellation request: when thread/task is ready to cooperate
- What to do after cancellation handling: up to receiver

Maybe useful in single-threaded⁶ and in multi-threaded⁷ environment.

⁶ <https://developer.mozilla.org/en-US/docs/Web/API/AbortSignal>

⁷ <https://learn.microsoft.com/en-us/dotnet/api/system.threading.cancellationtoken>

Using cancellation token for threads

```
void main() {  
    var token = new CancellationToken();  
    new Thread(() -> {  
        while (!token.IsCanceled()) { doSomething(); }  
    }).start();  
    ...  
    token.cancel();  
}
```

Using cancellation token for tasks

```
void main() {  
    var token = new CancellationToken();  
    executor.execute(() -> {  
        while (!token.IsCanceled()) { doSomething(); }  
    });  
    ...  
    token.Cancel();  
}
```


Using cancellation token for loops

```
void worker() {  
    while (!token.isCanceled()) {  
        doSomething();  
    }  
}  
  
void doSomething() {  
    for (long i = 0; i < gazillion(); i++) {  
        compute(i);  
    }  
}
```

Using cancellation token for loops

```
void worker() {  
    while (!token.isCanceled()) {  
        doSomething(token);  
    }  
}  
  
void doSomething(Cancellation token) {  
    for (long i = 0; i < gazillion(); i++) {  
        if (token.isCanceled()) { throw new CancelledException(); }  
        compute(i);  
    }  
}
```

Using cancellation token for loops

```
void worker() {  
    while (!token.isCanceled()) {  
        doSomething(token);  
    }  
}  
  
void doSomething(Cancellation token) {  
    for (long i = 0; i < gazillion(); i++) {  
        if (token.isCanceled()) { throw new CancelledException(); }  
        compute(i);  
    }  
}  
  
void compute(long i) {  
    Thread.sleep(i);  
}
```

Cancellation token

- easy to implement in any language
- straightforward implementation and behaviour
- supports both **thread cancellation** and **task cancellation**
- languages **with exceptions** suffer (`CancelledException` from almost any method)

Cancellation token

- easy to implement in any language
- straightforward implementation and behaviour
- supports both **thread cancellation** and **task cancellation**
- languages **with exceptions** suffer (`CancelledException` from almost any method)
- manual polling everywhere
 - before *any* blocking operation (synchronization, I/O)
 - in long loops or deep recursion
- requires standard library/external libraries support
- bad composability (how many cancellation tokens should be passed to function `foo`?)
- languages **without exceptions** suffer (which value to return if cancellation requested)

Cancellation token

- easy to implement in any language
- straightforward implementation and behaviour
- supports both **thread cancellation** and **task cancellation**
- languages **with exceptions** suffer (`CancelledException` from almost any method)
- manual polling everywhere
 - before *any* blocking operation (synchronization, I/O)
 - in long loops or deep recursion
- requires standard library/external libraries support **for better responsiveness**
- bad composability (how many cancellation tokens should be passed to function `foo`?)
- languages **without exceptions** suffer (which value to return if cancellation requested)

Question time

Question: How many CancellationTokens should I pass to `Thread.sleep`?



Lecture plan

1 Thread-safe counters

2 Cancellation

- Cancelling Future
- Forced cancellation
- Cooperative cancellation: cancellation token

3 Interruption

- Interruption points
- Design decisions
- Timeouts

4 Summary

Question time

Question: Imagine that we could associate a single cancellation token with every thread. Which parts of standard library should you adapt to support such token?



Language support for interruption: long methods

Common "long" methods:

- `Thread.sleep`
- File I/O
- User input
- Networking

Language support for interruption: long methods

Common "long" methods:

- `Thread.sleep`
- File I/O
- User input
- Networking
- User-specified functions

Language support for interruption: long methods

Common "long" methods:

- `Thread.sleep`
- File I/O
- User input
- Networking
- User-specified functions

Overview:

- Some methods are marked as "interruption points"

Language support for interruption: long methods

Common "long" methods:

- `Thread.sleep`
- File I/O
- User input
- Networking
- User-specified functions

Overview:

- Some methods are marked as "interruption points"
- Thread A prepares to handle interrupt requests (long method may return "exceptionally")

Language support for interruption: long methods

Common "long" methods:

- `Thread.sleep`
- File I/O
- User input
- Networking
- User-specified functions

Overview:

- Some methods are marked as "interruption points"
- Thread A prepares to handle interrupt requests (long method may return "exceptionally")
- Thread B requests interrupt which will be **eventually** delivered to Thread A.

Language support for interruption: long methods

Common "long" methods:

- `Thread.sleep`
- File I/O
- User input
- Networking
- User-specified functions

Overview:

- Some methods are marked as "interruption points"
- Thread A prepares to handle interrupt requests (long method may return "exceptionally")
- Thread B requests interrupt which will be **eventually** delivered to Thread A.

Concurrency is hard: "interrupted" task may finish with success

Language support for interruption

Advantages:

- similar to `CancellationToken`
- responsiveness due to standard library support
 - proper API design (throws `InterruptedException`)
 - low-level APIs under VM control (IO, networking)
- fine-grained control of cancellation handling
 - controlled frequency of polling points
 - easy to organize "uninterruptible sections"

Disadvantages:

- similar to `CancellationToken`
- still a lot of manual annotations and headache with external libraries
- receiver of interrupt does not know the sender/reason

Important: language-level cancellation API works for `Threads`

Question time

Question: Enter into synchronized section should be interruptible or not interruptible? What about `Object.wait`?



Language support for interruption: Java

Common "long" methods:

- `Thread.sleep`, File I/O, User input, Networking
- User-specified "interruption state checks"⁸

⁸ [https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#isInterrupted\(\)](https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#isInterrupted())

Language support for interruption: Java

Common "long" methods:

- `Thread.sleep`, File I/O, User input, Networking
- User-specified "interruption state checks"⁸
- Some synchronization (`Lock.lock`, `Condition.await`, `Object.wait`)

⁸ [https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#isInterrupted\(\)](https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#isInterrupted())

Language support for interruption: Java

Common "long" methods:

- `Thread.sleep`, File I/O, User input, Networking
- User-specified "interruption state checks"⁸
- Some synchronization (`Lock.lock`, `Condition.await`, `Object.wait`)

Ability to interrupt executing thread⁹

⁸ [https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#isInterrupted\(\)](https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#isInterrupted())

⁹ [https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#interrupt\(\)](https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#interrupt())

Language support for interruption: Java

Common "long" methods:

- `Thread.sleep`, File I/O, User input, Networking
- User-specified "interruption state checks"⁸
- Some synchronization (`Lock.lock`, `Condition.await`, `Object.wait`)

Ability to interrupt executing thread⁹

The most unfortunate naming for method: `static boolean Thread.interrupted`¹⁰

- checks that **current** thread
- was interrupted and
- clears interrupted status

⁸ [https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#isInterrupted\(\)](https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#isInterrupted())

⁹ [https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#interrupt\(\)](https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#interrupt())

¹⁰ [https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#interrupted\(\)](https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#interrupted())

Language support for interruption: Java

Common "long" methods:

- `Thread.sleep`, File I/O, User input, Networking
- User-specified "interruption state checks"⁸
- Some synchronization (`Lock.lock`, `Condition.await`, `Object.wait`)

Ability to interrupt executing thread⁹

The most unfortunate naming for method: `static boolean Thread.interrupted`¹⁰

- checks that **current** thread
- was interrupted and
- clears interrupted status

Remember, Future could return one of the following results:

- `Success<T>`, `Failure<Exception>`, `Cancellation<Exception>`,
`InterruptedException<Exception>`

⁸ [https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#isInterrupted\(\)](https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#isInterrupted())

⁹ [https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#interrupt\(\)](https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#interrupt())

¹⁰ [https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#interrupted\(\)](https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Thread.html#interrupted())

Interruptible vs. uninterruptible methods

<https://github.com/Svazars/parallel-programming/tree/main/hw/block1/5.2>

Homework

Task 5.2 Write a program that empirically checks if the following methods are interruptible:

- *Thread.sleep*
- *ReentrantLock.lock*
- *monitor enter in synchronized*
- *Condition.await*
- *Object.wait*
- *System.in.read*
- *FileWriter.write*

Task vs. thread interruption

Summary:

- User-defined `CancellationToken`
 - works for threads and for tasks
 - has limited responsiveness
- Language-designed thread interruption points
 - work only for threads
 - allow to quickly respond from almost any code

Question time

Question: Suppose I can interrupt threads. How to implement interruption of tasks?



Task vs. thread interruption

Let's implement task interruption on top of thread interruption

Task vs. thread interruption

Let's implement task interruption on top of thread interruption

- Running task should know current "carrier" thread (Executor internals)
- `InterruptedException` should be properly cleared from worker thread to not affect subsequent tasks
- No race conditions on interruptions allowed (spurious interrupts are forbidden)
- Handle interruption status for not-yet-started and already-finished tasks
- Be ready for successfully finished interrupted tasks

Task vs. thread interruption

Let's implement task interruption on top of thread interruption

- Running task should know current "carrier" thread (Executor internals)
- `InterruptedException` should be properly cleared from worker thread to not affect subsequent tasks
- No race conditions on interruptions allowed (spurious interrupts are forbidden)
- Handle interruption status for not-yet-started and already-finished tasks
- Be ready for successfully finished interrupted tasks

Concurrency is hard: making your own `Executor/ThreadPool` is not easy

Language support for interruption: other managed languages

Different languages use various policies for cancellation/interruption in standard library.

Language support for interruption: other managed languages

Different languages use various policies for cancellation/interruption in standard library. The most elaborate and widely adopted designs belong to languages with lightweight threading:

- Go ¹¹, managed language without exceptions
- Kotlin ¹², managed JVM-based language with compiler-driven CPS transformation (suspend methods)

¹¹<https://go.dev/blog/context>

¹²<https://kotlinlang.org/docs/cancellation-and-timeouts.html#asynchronous-timeout-and-resources>

Language support for interruption: other managed languages

Different languages use various policies for cancellation/interruption in standard library.

The most elaborate and widely adopted designs belong to languages with lightweight threading:

- Go ¹¹, managed language without exceptions
- Kotlin ¹², managed JVM-based language with compiler-driven CPS transformation (suspend methods)

We will cover goroutines/koroutines/coroutines in Lecture 13.

¹¹<https://go.dev/blog/context>

¹²<https://kotlinlang.org/docs/cancellation-and-timeouts.html#asynchronous-timeout-and-resources>

Language support for interruption: other managed languages

Different languages use various policies for cancellation/interruption in standard library. The most elaborate and widely adopted designs belong to languages with lightweight threading:

- Go ¹¹, managed language without exceptions
- Kotlin ¹², managed JVM-based language with compiler-driven CPS transformation (suspend methods)

We will cover goroutines/koroutines/coroutines in Lecture 13.

Basics you should know:

- Forced cancellation leads to corruption and should not be used in any language
- Cooperative cancellation:
 - could be totally user-designed (`CancellationToken`) with responsiveness trade-off
 - could be "embedded" into standard library (`Thread.interrupt`) with complexity trade-off
 - requires additional effort to guarantee responsiveness and consistency
 - widely adopted but slightly different in modern production languages

¹¹<https://go.dev/blog/context>

¹²<https://kotlinlang.org/docs/cancellation-and-timeouts.html#asynchronous-timeout-and-resources>

Question time

Question: It looks like any language could implement cooperative cancellation in the same manner: use thread-specific `CancellationToken` with `.interrupt` and `.isInterrupted`. Does Garbage Collection simplify anything here?



Lecture plan

1 Thread-safe counters

2 Cancellation

- Cancelling Future
- Forced cancellation
- Cooperative cancellation: cancellation token

3 Interruption

- Interruption points
- **Design decisions**
- Timeouts

4 Summary

Propagating cancellation

Thread A started Thread B and Thread C.

- Thread A was interrupted, should B and C be interrupted too? (parent -> child)
- Thread C was interrupted, should A be interrupted too? (child -> parent)
- Thread C was interrupted, should B be interrupted too? (sibling -> sibling)

Propagating cancellation

Thread A started Thread B and Thread C.

- Thread A was interrupted, should B and C be interrupted too? (parent -> child)
- Thread C was interrupted, should A be interrupted too? (child -> parent)
- Thread C was interrupted, should B be interrupted too? (sibling -> sibling)

It is up to you!

Use structured concurrency techniques and life cycle management (e.g. `ExecutorService.shutdown`).

Propagating cancellation

Thread A started Thread B and Thread C.

- Thread A was interrupted, should B and C be interrupted too? (parent -> child)
- Thread C was interrupted, should A be interrupted too? (child -> parent)
- Thread C was interrupted, should B be interrupted too? (sibling -> sibling)

It is up to you!

Use structured concurrency techniques and life cycle management (e.g. `ExecutorService.shutdown`).

Kotlin/Go contexts are really well-designed!

Ignoring cancellation

- Intended policy of task designer
- Task designer was not aware about interruption scenarios

Ignoring cancellation

- Intended policy of task designer
- Task designer was not aware about interruption scenarios

In general, it is good to write `interruption-aware` code to make it reusable and modular.

Ignoring cancellation

- Intended policy of task designer
- Task designer was not aware about interruption scenarios

In general, it is good to write `interruption-aware` code to make it reusable and modular. However, correct programs may ignore cancellation as soon as it affects performance only, not safety nor progress.

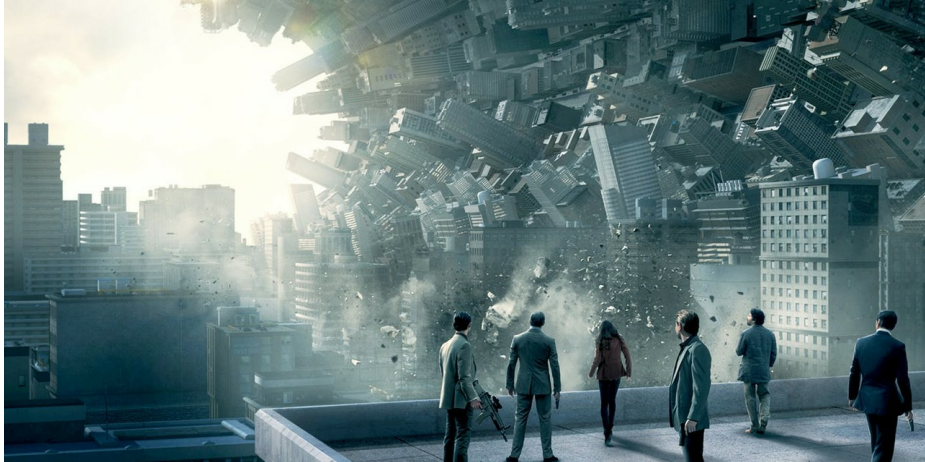
Cancelling cancellation

Cancelling cancellation

Do not play mind games!

Cancelling cancellation

Do not play mind games!



Notification and Interruption ordering

```
static Object monitor = new Object();  
void worker1()      { synchronized(monitor) { monitor.wait(); } }  
void worker2()      { synchronized(monitor) { monitor.wait(); } }  
void notifier()     { synchronized(monitor) { monitor.notify(); } }  
void interrupter() { getWorker1().interrupt(); }
```

Notification and Interruption ordering

```
static Object monitor = new Object();  
void worker1()      { synchronized(monitor) { monitor.wait(); } }  
void worker2()      { synchronized(monitor) { monitor.wait(); } }  
void notifier()     { synchronized(monitor) { monitor.notify(); } }  
void interrupter() { getWorker1().interrupt(); }
```

Language-level guarantees

- Delivered notify or delivered interrupt, no "overriding"
- Delivered interrupt could change Thread interruption status only (no immediate InterruptedException)

Notification and Interruption ordering

```
static Object monitor = new Object();  
void worker1()      { synchronized(monitor) { monitor.wait(); } }  
void worker2()      { synchronized(monitor) { monitor.wait(); } }  
void notifier()     { synchronized(monitor) { monitor.notify(); } }  
void interrupter() { getWorker1().interrupt(); }
```

Language-level guarantees

- Delivered notify or delivered interrupt, no "overriding"
- Delivered interrupt could change Thread interruption status only (no immediate InterruptedException)

It starts to be really hard!

Your tasks should be always aware of interruption requests. It helps to ensure your code is correct, responsiveness is not main goal here.

Lecture plan

1 Thread-safe counters

2 Cancellation

- Cancelling Future
- Forced cancellation
- Cooperative cancellation: cancellation token

3 Interruption

- Interruption points
- Design decisions
- Timeouts

4 Summary

Timeouts

Many blocking methods have optional `timeout` parameter:

- `Thread.join`
- `Lock.tryLock`
- `Condition.await`
- `Object.wait`

Timeouts

Many blocking methods have optional `timeout` parameter:

- `Thread.join`
- `Lock.tryLock`
- `Condition.await`
- `Object.wait`

Timeouts could "race" with timings, notifications and interruptions.

Timeouts

Many blocking methods have optional `timeout` parameter:

- `Thread.join`
- `Lock.tryLock`
- `Condition.await`
- `Object.wait`

Timeouts could "race" with timings, notifications and interruptions.

Unlike notifications (I am done!/I failed!) or interruptions (I stopped!), timeouts do not mean that executor stopped to consume resources (CPU, RAM, file handles...).

Timeouts

Many blocking methods have optional timeout parameter:

- `Thread.join`
- `Lock.tryLock`
- `Condition.await`
- `Object.wait`

Timeouts could "race" with timings, notifications and interruptions.

Unlike notifications (I am done!/I failed!) or interruptions (I stopped!), timeouts do not mean that executor stopped to consume resources (CPU, RAM, file handles...).

Remember, `Future.get(timeout)` could return one of the following results:

- `Success<T>`
- `Failure<Exception>`
- `Cancellation<Exception>`
- `Interruption<Exception>`
- `Timeout<Exception>`

Cancellation and interruption

Conclusion

Cancellation of not-yet-started tasks

- Easier to implement via concurrent state machine
- Saves resources if `cancel` is cheaper than finishing the task

Forced interruption

- is evil. Never use it!

Cooperative interruption

- May "lag", could be ignored by receiver
- Requires language/standard library support for better responsiveness
- Tricky because of many corner cases
 - Normal completion, Erroneous completion, Cancellation
 - Interruption, Notification, Timeout
 - Concurrent repeated interruption

Summary

- Cancellation of concurrently running tasks is useful to save computation resources
- Proper interruption requires cooperation
- There exist language-independent interruption designs and low-level language-specific ones
- Task interruption could be built on top of thread interruption
- Timeouts could be treated as a variation of "self-interruption"
- Developing safe and efficient concurrent systems is hard, consider using concurrent patterns

Summary: homework

Lecture 5, Tasks 5.*: <https://github.com/Svazars/parallel-programming/blob/main/hw/block1>